

SOFTWARE ENGINEERING

SOFTWARE REQUIREMENTS SPECIFICATION

for

Voice Assistant for Multimedia Controls

Version 1.0 – Approved

Prepared by: Group 12

PES University

September 2026

Team Members

Sl. No.	Name	SRN
1	Prajwal GT	PES2UG25CS818
2	Shreedhar R	PES2UG25CS823
3	Shrinivas S J	PES2UG24CS494
4	Sarang S Nair	PES2UG24CS447

Table of Contents

Revision History

1. Introduction

1.1 Purpose

1.2 Intended Audience and Reading Suggestions

1.3 Product Scope

1.4 References

2. Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 User Classes and Characteristics

2.4 Operating Environment

2.5 Design and Implementation Constraints

2.6 Assumptions and Dependencies

3. External Interface Requirements

3.1 User Interfaces

3.2 Software Interfaces

3.3 Communications Interfaces

3.4 Hardware Interfaces

4. Analysis Models

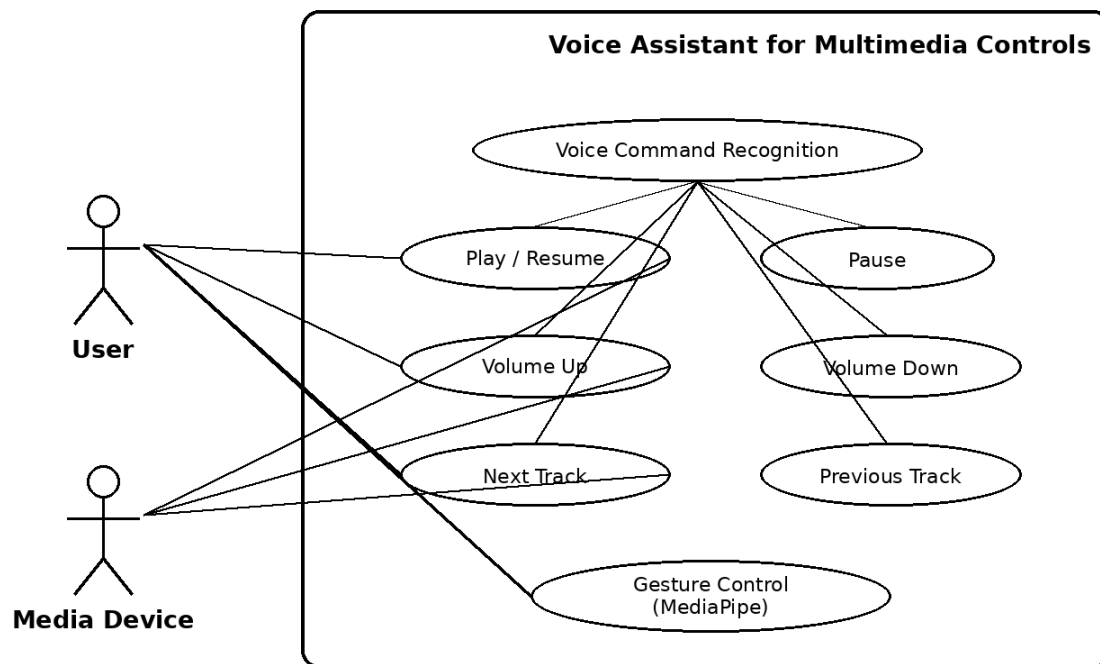


Figure 1: Use Case Diagram – Voice Assistant for Multimedia Controls

5. System Features

5.1 Voice Command Multimedia Control

5.2 Gesture-Based Multimedia Control

6. Other Nonfunctional Requirements

6.1 Performance Requirements

6.2 Safety Requirements

6.3 Security Requirements

6.4 Software Quality Attributes

6.5 Business Rules

7. Other Requirements

Appendix A: Glossary

Appendix B: Field Layouts

Appendix C: Requirement Traceability Matrix

Revision History

Name	Date	Reason for Changes	Version
Group 12 – Prajwal GT, Shreedhar R, Shrinivas S J, Sarang S Nair	September 2026	Initial SRS prepared for the proposed project	1.0
Group 12	September 2026	Added detailed functional/nonfunctional requirements, use cases and analysis model	1.0

1. Introduction

1.1 Purpose

This SRS specifies the requirements for the Voice Assistant for Multimedia Controls, Version 1.0. The system provides hands-free multimedia control using predefined voice commands through a microphone and a Raspberry Pi/ESP-based controller with Python software. Optional MediaPipe gesture recognition is included as an additional control mechanism.

1.2 Intended Audience and Reading Suggestions

This document is intended for the four project team members, faculty/evaluators, developers, testers and future maintainers. Sections 1–2 introduce the system and its context; Section 3 defines interfaces; Section 4 presents the use case analysis model; Section 5 defines system features and functional requirements; Section 6 defines nonfunctional requirements; Section 7 and the appendices cover additional requirements, glossary, field layouts and traceability.

1.3 Product Scope

The system enables a user to control multimedia playback without directly operating a keyboard, mouse or touchscreen. Core operations are play/resume, pause, volume up, volume down, next and previous. Selected hand gestures may also be mapped to multimedia actions using MediaPipe. The project aims to provide responsive, reliable, simple and modular multimedia control.

1.4 References

- Group 12 Project Proposal – Voice Assistant for Multimedia Controls, Version 1.0, September 2026.
- Project requirements and design discussions by Group 12.
- Documentation for the selected Raspberry Pi/ESP hardware platform.
- Python and selected voice-processing library documentation.
- MediaPipe documentation, if gesture recognition is implemented.

2. Overall Description

2.1 Product Perspective

This is a new, self-contained hardware-software prototype. The microphone captures voice input; the processing layer identifies the predefined command; and the multimedia-control layer sends the corresponding action to the connected computer or media device. An optional camera provides input for MediaPipe gesture recognition.

2.2 Product Functions

- Capture and recognize predefined voice commands.
- Play and resume multimedia.
- Pause currently playing multimedia.
- Increase and decrease media volume.
- Navigate to next and previous media items.
- Provide feedback after command recognition.
- Safely ignore unsupported or unrecognized input.
- Optionally recognize configured hand gestures using MediaPipe.

2.3 User Classes and Characteristics

User Class	Characteristics
General Multimedia User	Uses music, video or presentation applications; requires no specialized technical knowledge.
Student User	Frequently uses multimedia on computers and benefits from hands-free operation.
Project Administrator / Maintainer	Configures commands, hardware and software; performs testing and maintenance.

2.4 Operating Environment

Hardware: Raspberry Pi or ESP-based controller, microphone, optional camera and target computer/media device.

Software: Python, selected voice-processing libraries, operating-system/media-player control interface and MediaPipe when gesture control is enabled. The system shall operate in the supported environment of the selected hardware.

2.5 Design and Implementation Constraints

- Processing and memory are constrained by the selected Raspberry Pi/ESP platform.
- Microphone quality and background noise can affect voice recognition.
- The target media device must support the required control interface.
- The implementation will use Python and selected compatible libraries.
- Gesture control depends on camera availability and MediaPipe compatibility.
- The initial release is limited to predefined multimedia commands.

2.6 Assumptions and Dependencies

- A compatible microphone, controller and target media device are available.
- The required software libraries, drivers and operating-system interfaces are available.
- The user speaks supported commands clearly enough for recognition.
- Optional gesture control depends on a compatible camera and MediaPipe environment.
- Temporary communication failures may occur and must be handled safely.

3. External Interface Requirements

3.1 User Interfaces

The primary interface is voice-based. The user speaks a predefined command and the system executes the corresponding multimedia operation. Optional visual or audio feedback indicates whether the command was recognized. When gesture mode is enabled, the user can perform configured hand gestures.

3.2 Software Interfaces

Interface	Purpose
Python Application	Processes input and coordinates multimedia actions.
Voice Recognition Component	Converts captured speech into a recognized command.
Operating System / Media Player Control	Receives play, pause, volume and navigation actions.
MediaPipe (Optional)	Recognizes configured hand gestures.

3.3 Communications Interfaces

The controller may communicate with the target computer/media device through the selected wired or wireless connection. The communication mechanism shall use the protocol/interface supported by the chosen implementation and shall handle temporary connection failures without unintended actions.

3.4 Hardware Interfaces

Hardware	Interface Requirement
Microphone	Captures user voice input.
Raspberry Pi / ESP	Processes input and interfaces with the target device.
Camera (Optional)	Provides video input for MediaPipe gesture recognition.
Computer / Media Device	Receives multimedia control actions.

4. Analysis Models

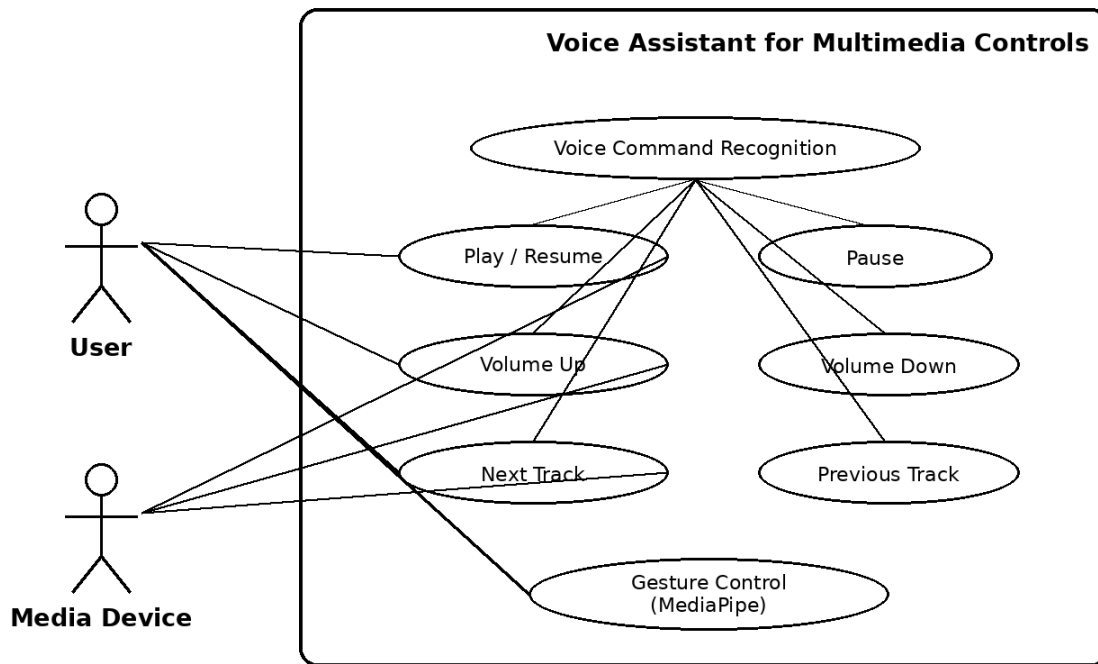


Figure 1: Use Case Diagram – Voice Assistant for Multimedia Controls

The following UML use case diagram identifies the main actor, external media device and multimedia-control use cases. The User initiates voice commands or optional gestures; the system performs the mapped action on the Media Device.

Figure 1: Use Case Diagram – Voice Assistant for Multimedia Controls

5. System Features

5.1 Voice Command Multimedia Control

Description and Priority: High. This is the core system feature and supports the primary multimedia operations through predefined voice commands.

5.1.1 Stimulus/Response Sequences

1. User speaks a supported command. 2. Microphone captures the input. 3. Voice-processing component recognizes the command. 4. System executes the corresponding multimedia action. 5. System provides feedback, if enabled, and returns to the ready state.

5.1.2 Functional Requirements

Requirement ID	Functional Requirement
REQ-VC-01	The system shall capture voice input through the configured microphone.
REQ-VC-02	The system shall recognize supported predefined voice commands.
REQ-VC-03	The system shall increase media volume for a volume-up command.
REQ-VC-04	The system shall decrease media volume for a volume-down command.
REQ-VC-05	The system shall pause currently playing media for a pause command.

Requirement ID	Functional Requirement
REQ-VC-06	The system shall play or resume media for a play/resume command.
REQ-VC-07	The system shall move to the next media item for a next command.
REQ-VC-08	The system shall move to the previous media item for a previous command.
REQ-VC-09	The system shall provide feedback after a supported command is recognized.
REQ-VC-10	The system shall ignore unsupported or unrecognized commands without causing unintended media actions.

5.2 Gesture-Based Multimedia Control

Description and Priority: Medium. This optional feature uses MediaPipe to recognize configured hand gestures and map them to selected multimedia actions.

5.2.1 Stimulus/Response Sequences

1. User enables gesture control. 2. Camera captures the user's hand. 3. MediaPipe processes the image. 4. System identifies a configured gesture. 5. The mapped multimedia action is executed. 6. System returns to the ready state.

5.2.2 Functional Requirements

Requirement ID	Functional Requirement
REQ-G-01	The system shall optionally capture camera input when gesture control is enabled.
REQ-G-02	The system shall recognize configured hand gestures using MediaPipe.
REQ-G-03	The system shall map each recognized configured gesture to its assigned multimedia action.
REQ-G-04	The system shall avoid triggering an action for an unsupported or uncertain gesture.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- The system should execute a recognized command with minimal noticeable delay.
- The voice-processing and multimedia-control components should remain responsive during continuous operation.
- Gesture processing, when enabled, should provide sufficiently responsive control for normal user interaction.

6.2 Safety Requirements

- The system shall avoid unintended multimedia actions when input is uncertain.
- The hardware shall be operated using suitable power supplies and normal electrical precautions.
- No physical safety certification is required for the academic prototype.

6.3 Security Requirements

- Only recognized supported commands or configured gestures shall trigger actions.
- Voice input shall not be unnecessarily stored.
- If an external voice-recognition service is used, its privacy and data-handling requirements shall be considered.
- System configuration should be protected from unauthorized modification where applicable.

6.4 Software Quality Attributes

Quality Attribute	Requirement
Usability	Commands and gestures should be simple, consistent and easy to remember.
Reliability	System should operate continuously without frequent crashes.
Accuracy	Supported voice commands and configured gestures should be recognized correctly under normal conditions.
Maintainability	Voice processing, gesture recognition and media control should be modular.
Portability	Prototype should run on the selected Raspberry Pi/ESP platform.
Robustness	Invalid input, background noise and temporary connection failures should be handled gracefully.
Testability	Each command and gesture should be independently testable.
Privacy	Voice input should not be unnecessarily stored or transmitted.

6.5 Business Rules

- Only predefined and configured commands shall control the multimedia device.
- Unsupported input shall not change the media state.
- Each gesture, when enabled, shall correspond to one configured multimedia action.
- All team members shall participate in major project activities, with primary product ownership divided among the four members.

7. Other Requirements

- No database is required for the core prototype.
- The design should permit future addition of commands, languages, applications and IoT devices.
- The implementation shall document required hardware, software libraries and configuration steps.
- Any third-party component used shall be compatible with the selected project environment.

Appendix A: Glossary

Term	Definition
Voice Command	A predefined spoken instruction used to control multimedia.
Multimedia Control	An operation such as play, pause, volume change or track navigation.
Raspberry Pi	A small single-board computer that may be used as the hardware controller.
ESP	A family of microcontroller platforms that may be used for hardware control.
MediaPipe	A framework used here optionally for hand-gesture recognition.
SRS	Software Requirements Specification.

Term	Definition
SRN	Student Registration Number.
User	Person issuing voice commands or configured gestures.

Appendix B: Field Layouts

The core Voice Assistant for Multimedia Controls does not require customer-registration or transactional database fields. The following configuration field layout applies to the command mapping maintained by the system.

Field	Length	Data Type	Description	Mandatory
Command ID	10	Alphanumeric	Unique identifier for a configured command.	Y
Voice Command	50	String	Predefined phrase recognized by the system.	Y
Action	30	String	Multimedia action mapped to the command.	Y
Gesture ID	20	Alphanumeric	Identifier for an optional configured gesture.	N
Enabled	5	Boolean	Indicates whether the command/gesture is active.	Y
Feedback	50	String	Optional feedback message or indication.	N

Report requirement: The system may provide a command-status report containing Command ID, Voice Command, Action, Enabled status and recognition/test status.

Appendix C: Requirement Traceability Matrix

Sl. No.	Requirement ID	Brief Description	Architecture Reference	Design/Code Reference	Test Case ID	System Test ID
1	REQ-VC-01	Capture voice input	Voice Input Module	voice_input.py	TC-VC-01	ST-01
2	REQ-VC-02	Recognize predefined commands	Command Recognition Module	command_recognition.py	TC-VC-02	ST-02
3	REQ-VC-03	Volume up	Multimedia Control Module	media_control.py	TC-VC-03	ST-03
4	REQ-VC-04	Volume down	Multimedia Control Module	media_control.py	TC-VC-04	ST-04
5	REQ-VC-05	Pause media	Multimedia Control Module	media_control.py	TC-VC-05	ST-05
6	REQ-VC-06	Play/resume media	Multimedia Control Module	media_control.py	TC-VC-06	ST-06
7	REQ-VC-07	Next media	Multimedia Control Module	media_control.py	TC-VC-07	ST-07

Sl. No.	Requirement ID	Brief Description	Architecture Reference	Design/Code Reference	Test Case ID	System Test ID
8	REQ-VC-08	Previous media	Multimedia Control Module	media_control.py	TC-VC-08	ST-08
9	REQ-VC-09	Command feedback	Feedback Module	feedback.py	TC-VC-09	ST-09
10	REQ-VC-10	Ignore invalid commands	Command Recognition Module	command_recognition.py	TC-VC-10	ST-10
11	REQ-G-01	Capture gesture input	Gesture Module	gesture_control.py	TC-G-01	ST-11
12	REQ-G-02	Recognize gestures	MediaPipe Module	gesture_control.py	TC-G-02	ST-12
13	REQ-G-03	Map gesture to action	Gesture Mapping Module	gesture_control.py	TC-G-03	ST-13
14	REQ-G-04	Reject uncertain gestures	Gesture Module	gesture_control.py	TC-G-04	ST-14

Appendix D: Product Ownership Summary

Team Member	SRN	Primary Functional Ownership	Qualitative Contribution
Prajwal GT	PES2UG25CS818	Voice command detection; play/resume; pause	Improve voice recognition accuracy and reduce response delay.
Shreedhar R	PES2UG25CS823	Volume up/down; Raspberry Pi/ESP and device communication	Ensure reliable communication and consistent volume control.
Shrinivas S J	PES2UG24CS494	Next/previous; command feedback	Ensure reliable track navigation and clear feedback.
Sarang S Nair	PES2UG24CS447	MediaPipe gesture recognition; gesture mapping; integration	Improve gesture accuracy and reduce unintended triggers.

All four members are expected to participate in every major project activity.